

1.- Descripción de la tarea

MUY IMPORTANTE: A lo largo de toda la tarea "XYZ" hace referencia a las iniciales de tu nombre y apellidos. No debes dejar "XYZ", sino sustituirlo por tus iniciales.

Por ejemplo, para "Santiago Ruíz Valbuena" pondríamos "SRV" en vez de "XYZ".

Debes conocer

Usar ChatGPT o Gemini o Copilot o cualquier otra IA generativa es como sentarse en la mesa y ver como otros comen, es como tener sed y ver como otros beben agua, y ver como otros comen y como otros beben agua... no te quitará ni el hambre ni la sed. Es como querer aprender a montar en bicicleta viendo como otros montan en bicicleta.

En esta tarea se propone un acercamiento al patrón MVC de una forma sencilla, y vas a aplicar el patrón MVC sin usar ningún framework. En la siguiente unidad se introduce Laravel y conocer ahora el patrón MVC y la programación orientada a objetos te dará una mejor perspectiva de que es el patrón MVC, de lo que es un framework MVC y de por qué están contruidos de la forma en la que están contruidos. Al principio habrá conceptos que te costarán un poco de trabajo entender, pero luego todo coge forma y se hace más sencillo.

Para el propósito de esta ejercicio vamos a usar a usar el mismo esquema de base de datos de la tarea 3, no obstante te recomendamos crearlo en una base de datos diferente a la de la tarea 3:

```
CREATE TABLE IF NOT EXISTS `generos` (  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `nombre` VARCHAR(45) NOT NULL,  
  `descripcion` TINYTEXT NULL,  
  PRIMARY KEY (`id`))  
ENGINE = InnoDB;  
  
CREATE TABLE IF NOT EXISTS `peliculas` (  
  `id` BIGINT NOT NULL AUTO_INCREMENT PRIMARY KEY,
```

```

`titulo` VARCHAR(60) NOT NULL,
`genero` INT,
`direccion` VARCHAR(100) NOT NULL,
`duracion` INT NOT NULL CHECK (duracion>0 AND duracion<500),
`argumento` TINYTEXT NOT NULL,
`anio` INT NOT NULL CHECK (anio>1900),
INDEX `fk_peliculas_generos1_idx` (`genero` ASC),
CONSTRAINT `fk_peliculas_generos1`
    FOREIGN KEY (`genero`)
    REFERENCES `generos` (`id`)
    ON DELETE SET NULL
    ON UPDATE SET NULL )
ENGINE = InnoDB;

```

A continuación tienes una serie de datos de ejemplo para trabajar con las tablas anteriores:

```

INSERT INTO `generos` (`id`, `nombre`, `descripcion`) VALUES
    (1, 'animación', 'Películas de animación, tanto tradicionales como por computadora'),
    (2, 'drama', 'Películas centradas en el desarrollo emocional y conflictos de los personajes'),
    (3, 'ciencia ficción/fantasía', 'Películas con elementos tecnológicos, futuristas o fantásticos'),
    (4, 'comedia', 'Películas destinadas a provocar risa y entretenimiento ligero'),
    (5, 'thriller', 'Películas de suspense y tensión'),
    (6, 'documental', 'Películas basadas en hechos reales y investigación');

INSERT INTO `peliculas` (`id`, `titulo`, `genero`, `direccion`, `duracion`, `argumento`, `anio`) VALUES
    (1, 'El viaje de Luma', 1, 'Carla Gómez', 95, 'Una joven descubre un mundo mágico durante un viaje inesperado.', 2018),
    (2, 'La caja de colores', 1, 'Miguel Ruiz', 82, 'Una aventura animada sobre amistad y creatividad.', 2015),
    (3, 'Bosque de papel', 1, 'Ana Torres', 100, 'Animación poética sobre un bosque que guarda secretos.', 2021),
    (4, 'Sombras en la ciudad', 2, 'Luis Martín', 120, 'Historia íntima de personajes cuyas vidas se entrelazan.', 2010),
    (5, 'Cartas que no llegaron', 2, 'María López', 110, 'Un drama sobre decisiones, pérdidas y segundas oportunidades.', 2017),
    (6, 'Día de otoño', 2, 'Pablo Herrera', 98, 'Retrato emocional de una familia en un día decisivo.', 2005),
    (7, 'Estrella de vidrio', 3, 'Elena Vega', 130, 'Ciencia ficción sobre un descubrimiento que cambia la humanidad.', 2023),
    (8, 'Aurora mecánica', 3, 'Diego Salas', 140, 'Una fusión de tecnología y mito en un futuro cercano.', 2020),
    (9, 'Los portales', 3, 'Nora Díaz', 105, 'Exploradores atraviesan mundos paralelos en busca de respuestas.', 2012),

```

(10, 'Un café y dos risas', 4, 'Raúl Sánchez', 85, 'Comedia ligera sobre enredos entre amigos en una cafetería.', 2011),

(11, 'Manual de excusas', 4, 'Sofía Ramos', 90, 'Situaciones cómicas derivadas de excusas cada vez más absurdas.', 2019),

(12, 'Vecinos a destiempo', 4, 'Óscar Medina', 95, 'Comedia sobre la convivencia forzada entre vecinos opuestos.', 2008),

(13, 'Silencio en la noche', 5, 'Carlos Núñez', 105, 'Un suspense tenso donde cada detalle cuenta.', 2016),

(14, 'La huella', 5, 'Irene Paredes', 115, 'Thriller sobre una investigación que desentierra viejos secretos.', 2014),

(15, 'Bajo la mira', 5, 'Fernando Cruz', 125, 'Persecución intelectual entre cazador y presa en clave psicológica.', 2022),

(16, 'Raíces', 6, 'Pilar Ortiz', 72, 'Documental que explora tradiciones y memoria colectiva.', 2002),

(17, 'Olas y gentes', 6, 'Andrés Molina', 80, 'Retrato documental de comunidades costeras y su relación con el mar.', 2015),

(18, 'Archivos olvidados', 6, 'Lucía Blanco', 95, 'Investigación documental sobre archivos y testimonios del pasado.', 1999);

Para desarrollar el ejercicio de forma más sencilla se proporciona la clase `Peticion` (no es obligatorio su uso). Se trata de una clase que contiene métodos para facilitar el acceso a los datos recibidos vía GET y POST, así como otros métodos que es muy conveniente que utilices. Puedes modificar la clase `Peticion` a tu gusto.

[Clase Peticion](#) (zip - 1,68 KB)

Se valorará en todos los casos la corrección ortográfica y gramatical de los mensajes para comunicarnos con el usuario, así como la presentación clara de cualquier información que se muestre al usuario.

EJERCICIO 01

En esta primera etapa vamos a instalar y configurar un proyecto con *composer*. *composer* es un gestor de dependencias para PHP que permite automatizar la carga de dependencias y de clases. Este apartado viene explicado en los contenidos.

En este ejercicio tienes que realizar lo siguiente:

- En primer lugar, instala *composer* siguiendo lo indicado en los contenidos.
- Crea el proyecto `dwes04` con *composer*.
- **Namespace de tu aplicación.** El namespace de tu aplicación será `xyz04`.
- **Directorio de clases de tu aplicación.** El directorio o carpeta en tu proyecto que contendrá las clases de creación propia pertenecientes al espacio de nombres `xyz04` será `dwes04/srcxyz`. Crea dicho directorio. Las clases que irás creando tendrán que ir cargándose por autoloading PSR-4, por lo que debes configurar de forma adecuada el archivo `composer.json` para autoloading PSR-4.
- Añade como dependencia de producción la versión 4.4.1 de Smarty.
- **Directorio donde se ubicarán las plantillas de Smarty.** El directorio donde se ubicarán las plantillas de Smarty

será `dwes04/plantillasxyz` dentro del proyecto, crea dicho directorio. El directorio usado para cache y plantillas compiladas de smarty serán `dwes04/tmp/compiled_templates` y `dwes04/tmp/smarty_cache`, crea ambos directorios.

Recomendación

Una vez realizado lo anterior, es recomendable (pero no obligatorio) poner en práctica lo aprendido en esta unidad. Para ponerlo en práctica puedes crear una clase de prueba a tu elección, un script de punto de entrada inicial (por ejemplo, `http://localhost/dwes04/prueba2.php`) y dos plantillas Smarty, una con un formulario para recoger datos y otra para mostrar los resultados. Este proceso puedes hacerlo con clases de tu invención, por ejemplo:

- Crea una clase de prueba (en el directorio y *namespace* esperado). Intenta que la clase tenga un constructor y al menos dos métodos de instancia (es decir, dos métodos no estáticos).
- Crea dos plantillas Smarty (una con un formulario y otra para mostrar la salida).
- Crea un script llamado `prueba1.php`, accesible en la ruta `http://localhost/dwes04/prueba1.php` que haga uso de la clase anterior (constructor y dos métodos de instancia) y de las plantillas Smarty. La clase creada deberá cargarse por autoloading PSR-4, así como Smarty.

EJERCICIO 02

IMPORTANTE

- En todo el proyecto se utilizará exclusivamente PDO para acceder a la base de datos.
- Los ejercicios no se darán por válidos si hay código para realizar operaciones con la base de datos fuera de las clases del modelo específicamente pensadas para ello. Es decir, todo el código para operar con la base de datos deben estar en los métodos de las clases donde debe.
- Los ejercicios no se darán por válidos si hay código para realizar operaciones que no use consultas preparadas y try-catch. Es decir, todo código que opere con la base de datos debe usar consultas preparadas (si tiene parámetros en la consulta) y try-catch.

Siguiendo las líneas del ejemplo explicado en la unidad en el apartado 5, vamos a crear el modelo de una aplicación que sigue el patrón MVC sencillo (sin utilizar ningún *framework* complejo). Recuerda que el modelo es el que da soporte a los datos que maneja la aplicación.

Es necesario que las clases del modelo las separes en un subespacio (`namespace xyz04/subespacio;`), tal y como explica los contenidos, donde

"subespacio" será un texto a tu elección. No debes dejar "subespacio" tal cual, tiene que ponerle otro nombre a ese subespacio. A ese directorio a partir de ahora lo llamaremos "directorio del modelo". Tienes que prever que las clases siempre se cargarán usando autoloading PSR-4. Solo el enumerado `DBResult` irá en un subespacio diferente, el resto de clases e interfaces del modelo irán ahí.

Los métodos del modelo no deben enviar texto al navegador ni establecer conexión con la base de datos, esas no son sus atribuciones.

No se hará *include* ni *require* de estas clases o interfaces en ningún momento. Como se indica antes, las clases e interfaces deben cargarse por autoloading usando el cargador PSR-4 de *composer*, por lo que debes ubicarlas en un directorio del modelo e indicar el subespacio de nombres de forma correcta.

En los métodos que trabajan con la base de datos:

- Usar **consultas preparadas** cuando haya parámetros para las consultas.
- Usar `try-catch`.

No usarlo se considerará un error grave.

Paso 1) Enumerado DBResult

En el espacio de nombres `xyz04/servicios` tienes que crear un enumerado llamado `DBResult`. Dado el espacio de nombres anterior y la configuración de proyecto indicada en el apartado 1.1 tendrás que crear el enumerado en el archivo `/dwes04/srcxyz/servicios/DBResult.php` (se proporciona a modo de ejemplo inicial). El contenido de dicho archivo será:

```
namespace DWES04SOL\servicios;

enum DBResult: int {
    case XYZ_DB_EXCEPTION = -1;
    case XYZ_DB_OPNOTFULFILLED = -2;
    case XYZ_DB_NOCOLS_AFFECTED = -3; //OPCIONAL
    case XYZ_DB_EMPTYRESULT = -4;
}
```

En las operaciones con las bases de datos (todas), si hay un error se retornará un valor `DBResult` adecuado a la situación que se ha producido. Es importante seleccionar el valor retornado adecuado.

Paso 2) Crea la clase EntidadIdentificableXYZ

Crea la clase `EntidadIdentificablexyz` que tendrá las siguientes características:

- Será una clase abstracta.

- Tendrá los siguientes atributos:
 - Un atributo protegido llamado `id` tipo `int` que puede ser nulo (`?int`). Su valor por defecto será `null`.
 - Un método protegido llamado `getId` que permitirá obtener el valor de dicho atributo.
 - Un método público llamado `setId` que permitirá establecer el valor de dicho atributo.

Paso 3) Crear la interfaz `IGuardableXYZ`.

La interfaz `IGuardableXYZ` tendrá 3 métodos:

- Método **guardar**. El método **guardar** será un método que tendrá como parámetro una conexión `pdo` a la base de datos y retornará obligatoriamente `DBResult|int`.
- Método **estático rescatar**. El método **rescatar** será un método que tendrá dos parámetros: una conexión `pdo` a la base de datos, y el identificador (`id`), que corresponde con la clave primaria en la tabla asociada. Retornará obligatoriamente `DBResult|EntidadIdentificableXYZ`.
- Método **estático borrar**. El método **borrar** será un método que también tendrá dos parámetros: una conexión `pdo` a la base de datos, y el identificador (`id`), que corresponde con la clave primaria en la tabla asociada. Retornará obligatoriamente `DBResult|int`.

El objetivo de estos métodos es unificar los métodos que se utilizan para gestionar los datos almacenados en la base de datos. En este caso solo usamos una tabla, pero cuando se usan muchas tablas tener métodos con los mismos nombres y parámetros facilita el trabajo.

Paso 4) Crea la interfaz `IListableXYZ`.

La interfaz `IListableXYZ` tendrá 2 métodos:

- Método **estático listar**. El método **listar** será un método que tendrá como parámetro una conexión `pdo` a la base de datos y retornará obligatoriamente `DBResult|array`.
- Método **estático existe**. El método **existe** será un método que tendrá dos parámetros: una conexión `pdo` a la base de datos, y el identificador (`id`), que corresponde con la clave primaria en la tabla asociada. Retornará obligatoriamente `DBResult|int`.

Paso 5) Crea la clase **Genero**.

En el directorio del modelo deberás crear la clase **Genero** que encapsulará la información de un género de película. Esta clase extenderá obligatoriamente la clase `EntidadIdentificableXYZ`. Además, esta clase tendrá:

- Un atributo privado llamado `$nombre`.

- Un método público para obtener el valor de `$nombre` (`getNombre`).
- Un método público para establecer el valor de `$nombre` (`setNombre`).

IMPORTANTE: no se añade el atributo "id" dado que dicho atributo ya está en la clase padre `EntidadIdentificableXYZ`

Paso 6) Crea la clase Generos.

En el directorio del modelo deberás crear la clase `Generos` que encapsulará operaciones sobre todo el conjunto de géneros. Esta clase implementará obligatoriamente la clase interfaz `IListableXYZ`. A la hora de implementar la interfaz ten en cuenta lo siguiente:

- La función `listar` retornará un array de instancias de la clase `Genero` donde estarán todos los géneros almacenados en la base de datos o un valor de `DBResult` adecuado. Cada instancia de la clase `Genero` contendrá la información de un genero de la base de datos. No será correcto retornar un array asociativo bidimensional directamente con `fetchAll` o equivalente.
- La función `existe` retornará 1 si el género existe, 0 si el género no existe o un valor `DBResult` adecuado en caso de que se produzca otra circunstancia.

Paso 7) Crear la clase Pelicula.

En el directorio del modelo deberás crear la clase `Pelicula`. Esta clase extenderá la clase `EntidadIdentificableXYZ` e implementará la interfaz `IGuardableXYZ`. Esta clase tendrá:

- Atributos **privados** para contener todos y cada uno de los datos de la tabla `PELICULAS` especificada en el apartado "Descripción de la tarea", teniendo en cuenta que el valor por defecto de todos los atributos será `null`.
- No tendrá el atributo `id`, dado que esto ya lo hereda de la clase padre.

Dados a dichos atributos, esta clase tendrá los siguientes métodos `get` y `set`:

- Método público `getId` y protegido `setId` no han de implementarse, ya existen en la clase padre.
- Métodos públicos tipo `get` para obtener el valor del resto de datos.
- Métodos públicos tipo `set` para establecer el resto de datos.

Aparte de los métodos anteriores hay que implementar los métodos propios de la interfaz:

- Implementación del método `guardar`. Este método guarda o actualiza el contenido en la base de datos asociado a esta clase excepto `id` que se genera automáticamente.
 - Situaciones:

- Si el atributo `id` es `null` es porque es una nueva película, por lo que se procede a guardar por primera vez y después se establecerá el valor del atributo `id` en la clase `Película`.
 - ¡Cuidado! El campo `id` es autogenerado por la base de datos, esto significa que al guardar por primera vez debe obtenerse el `id` autogenerado para después almacenar su valor en el atributo `id` de la clase usando el método `set` apropiado.
- Si el atributo `id` no es `null` es porque es una película ya existente, por lo que se procede a actualizar. En este caso no hay que modificar el valor del atributo `id` (dado que no cambia).
 - Como resultado de la ejecución de este método retornará el número de filas creadas o actualizadas en la base de datos, si todo ha ido bien.
- Implementación del método estático `rescatar`. La función de este método es obtener un registro con el `id` indicado y retornar una instancia de la clase con los datos rescatados. Si todo va bien retornará una instancia de la clase `Película` con todos los datos rellenos con la información procedente de la tabla `libros`.
- Implementación del método estático `borrar`. La función de este método es borrar la base de datos del registro con el `id` de la tabla indicado y como resultado de la ejecución se retornará el número de filas creadas o actualizadas en la base de datos.

Paso 8) Clase Películas.

Esta clase implementará obligatoriamente la clase interfaz `IListablexyz`. A la hora de implementar la interfaz ten en cuenta los mismos aspectos que se han descrito a la hora de crear la clase `Generos`. El objetivo es el mismo que en dicha clase, pero con otro conjunto de datos.

Puedes dejarlo para el final, pero en el Ejercicio 5 se pide que se pruebe el modelo creado. Es buena idea que le eches un vistazo al ejercicio 5 antes de pasar al ejercicio 3.

1.3.- Ejercicio 3: controladores y vistas (I)

IMPORTANTE

Como parte de la realización de este apartado tendrás que elaborar un vídeo de no más de 30 segundos en formato MP4 donde se muestre la funcionalidad aquí descrita en ejecución. No olvides que debe aparecer tu imagen de fondo en la

plataforma en una esquina. El vídeo de este apartado se deberá llamar videoXYZ_1.mp4.

En este apartado crearemos la clase controladora de las diferentes acciones que se podrán realizar en la aplicación web. La clase controladora se llamará ControladorXYZ.

Nuevamente, al igual que ocurría en el modelo, es necesario que la clase que agrupa los métodos para cada acción se separe en un subespacio diferente (namespace espacio/subespacio;), tal y como explica los contenidos. A este directorio lo llamaremos "directorio del controlador". Debes prever que esta clase será cargada por autoloading PSR-4 a través de *composer*.

Normalmente, tal y como se muestra en el apartado 5 de la unidad 4, las acciones a controlar se implementan con métodos estáticos, por ejemplo:

```
public static function accion_a_controlar(Peticion $p, PDO $p, Smarty $s)
{
    ...
    $s->display('template.tpl');
}
```

Antes de continuar recuerda que:

- Los controladores reciben lo que necesitan por parámetro (conexión PDO, instancia de Smarty, etc.). Esos recursos (conexión PDO, instancia de Smarty, instancia de clase Peticion, etc.) se crean en el enrutador.
- Los controladores no muestran salida por si mismos, sino que lo hacen a través de la vista (en este caso plantillas Smarty), incluidos los formularios.
- Los controladores no realizan consultas a la base de datos, eso lo hacen en las clases del modelo.
- Los controladores SI verifican los datos recibidos vía POST o GET antes de pasarlos al modelo.
- Los controladores interconectan los componentes.

Nota: no es obligatorio el uso de la clase *Peticion* proporcionada. En caso de no usarla, deberás filtrar adecuadamente los datos recibidos vía POST y GET donde corresponda con *filter_input*.

Una vez hecho lo anterior tienes que implementar el controlador por defecto, en el cual se mostrará un listado de películas y permitirá ordenarlo de forma ascendente o descendente por una o más columnas (al menos una).

- **Controlador por defecto.** Se mostrará un listado de películas ordenado donde se incluirán todos los datos de la base de datos para cada película (incluido id). Este controlador tendrá un comportamiento diferente si se reciben datos vía POST o si no se reciben datos vía POST:

- Caso A) NO se reciben datos vía POST o los datos recibidos son incorrectos: mostrará un listado de todas las películas existentes (sin ordenar, tal y como están en la base de datos) y un formulario para indicar si se desean ordenar por Título, Año o Duración de forma ascendente o descendente (al menos uno de los tres). El formulario tendrá como destino la ruta por defecto.
- Caso B) SI se recibe vía POST el orden (y opcionalmente la columna si se puede seleccionar más de una) los datos se mostrarán en función del orden indicado para la columna indicado. Si el orden recibido no es válido, se mostrará un mensaje de error y se listará como en el A). En cualquier caso, se volverá a mostrar el formulario para ordenar.
- Haz uso de sesiones para que en caso de volver a acceder a la ruta por defecto aparezca ya ordenado de la última forma que indicó el usuario y que aparezca preseleccionado en el formulario.

Lista de películas

[Ir a formulario para añadir nueva película](#)

Ordenar por:		Año	--Seleccione--	Ordenar		
Id	Título	Año	--Seleccione--	Dirección	Duración	Argumento
			Ascendente Descendente			
1	El viaje de Luma	2018		Carla Gómez	96	Una joven descubre un mundo mágico durante un viaje inesperado.
2	La caja de colores	2015	animación (1)	Miguel Ruiz	82	Una aventura animada sobre amistad y creatividad.
3	Bosque de papel	2021	animación (1)	Ana Torres	100	Animación poética sobre un bosque que guarda secretos.
4	Sombras en la ciudad	2010	drama (2)	Luis Martín	120	Historia íntima de personajes cuyas vidas se entrelazan.
5	Cartas que no llegaron	2017	drama (2)	María López	110	Un drama sobre decisiones, pérdidas y segundas oportunidades.
6	Día de otoño	2005	drama (2)	Pablo Herrera	98	Retrato emocional de una familia en un día decisivo.
7	Estrella de vidrio	2023	ciencia ficción/ fantasía (3)	Elena Vega	130	Ciencia ficción sobre un descubrimiento que cambia la humanidad.

RECUERDA: los métodos del controlador no hacen consultas SQL ni muestran datos con "echo" o similar. El acceso a los datos debe hacerse a través de los métodos del modelo y la generación de HTML y texto debe hacerse a través de plantillas Smarty. No seguir esta regla significa que el ejercicio no es correcto.

Paralelamente a la implementación del modelo deberás crear el enrutador (`dwes04/index.php`) y atender a la ruta por defecto: `http://localhost/dwes04/index.php`. En dicho enrutador deberás:

- Cargar el archivo `vendor/autoload.php` para que se produzca la carga automática de clases.

- Crear una instancia de PDO para pasarla al controlador por defecto si es necesario.
- Crear una instancia de Smarty para pasarla al controlador por defecto si es necesario.
- Ejecutar el controlador por defecto.

IMPORTANTE: todas las clases se cargarán por autoloading PSR-4, en ningún momento se realizará `include` o `require`. La única situación donde se podrá hacer uso de `include` o `require` será para cargar información de configuración de acceso a la base de datos desde el archivo `index.php`.

1.4.- Ejercicio 4: controladores y vistas (II)

Orientaciones para el alumnado

En este ejercicio se trabajan los siguientes criterios de evaluación del RA 5: *desarrolla aplicaciones Web identificando y aplicando mecanismos para separar el código de presentación de la lógica de negocio.*

- a) Se han identificado las ventajas de separar la lógica de negocio de los aspectos de presentación de la aplicación.
- b) Se han analizado y utilizado mecanismos y frameworks que permiten realizar esta separación y sus características principales.
- g) Se han aplicado los principios y patrones de diseño de la programación orientada a objetos.

IMPORTANTE

Como parte de la realización de este apartado tendrás que elaborar dos vídeos de no más de 30 segundos en formato MP4 donde se muestre la funcionalidad aquí descrita en ejecución. No olvides que debe aparecer tu imagen de fondo en la plataforma en una esquina. Los vídeos de este apartado se deberá llamar `videoXYZ_2.mp4` (funcionalidad `añadir` película) y `videoXYZ_3.mp4` (funcionalidad `borrar` película con confirmación).

Una vez que tienes implementada la acción por defecto en el controlador, vamos a implementar más acciones. Para cada nueva acción a realizar tendrás que:

1. Modificar el enrutador (`dwes04/index.php`) para capturar la nueva acción e invocar al método de la clase controladora correspondiente en función de si la petición es tipo GET o POST.
2. Crear los métodos adicionales en el controlador para cada nueva acción. Recuerda que a cada método hay que pasar todo lo necesario (instancia de la clase PDO, instancia de la clase Smarty, etc.).
3. Dentro de cada método del controlador:

1. Se tendrán que revisar los datos recibidos vía GET o POST según corresponda, validarlos y sanearlos.
2. Hacer las operaciones con la base de datos pertinentes, usando las clases del modelo, nunca se harán operaciones con la base de datos directamente en el método controlador de la acción.
3. Se mostrará un resultado al usuario siempre a través de plantillas Smarty.

Las funcionalidades a implementa son las dos siguientes

MOSTRA FORMULARIO PARA AÑADIR PELICULAS

Cuando se acceda a la ruta `?accion=nueva_pelicula_form_XYZ` se mostrará un formulario con los datos necesarios para una nueva película (excepto id que es autogenerado por MySQL).

En el listado de películas (mostrado por defecto) habrá un enlace a la ruta anterior que permitirá acceder al formulario para añadir una nueva película:

REALIZAR ACCION DE GUARDAR EL NOMBRE

El formulario anterior se enviará vía POST a la ruta `?accion=nueva_pelicula_guardar_XYZ`. El enrutador (`index.php`) solo ejecutará el método controlador de la acción en caso de que sea POST, si no es POST se ejecutará otro método controlador donde se mostrará un error en la acción.

La acción de guardar será tratada en método de la clase controladora propio (método controlador de acción específico), donde se comprobará que los datos recibidos son correctos y conforme a lo esperado, y en caso de que lo sean, dará de alta la película y se mostrará el id de la película recién creada.

En caso de que los datos recibidos no sean correctos o no se proporcionen todos los datos necesarios, se mostrará un mensaje de error apropiado para cada error encontrado.

IMPORTANTE: en la tabla todos los campos son NOT NULL (es decir, no pueden estar vacíos). Debes verificar que los datos correspondan con el tipo esperado para la tabla proporcionada (ya sea numérico o cadena de texto), que el año esté comprendido entre 1965 y el año actual, que la duración sea un número mayor de cero y menor o igual a 500, que el género exista y que los campos de texto no excedan el tamaño previsto en la tabla.

FORMULARIO PARA BORRAR UNA PELICULA

Para cada película se añadirá una acción que permitirá borrarla en el listado de películas. Esta acción será tipo POST y llevará a la ruta ?accion=borrar_pelicula_form_XYZ.

La acción ?accion=borrar_pelicula_form_XYZ requerirá la implementación de un método controlador de la acción independiente que mostrará un formulario de confirmación siguiendo las directrices generales ya explicadas. El enrutador se encargará de invocar a dicho método controlador solo si el método HTTP es POST, si el método HTTP no es post se ejecutará otro método controlador donde se mostrará un error en la acción.

El formulario de confirmación generado tendrá un aspecto como el siguiente (donde no debe usarse bajo ningún concepto required en el checkbox). El formulario de confirmación será POST y llevará a la ruta ?accion=borrar_pelicula_confirmacion_XYZ donde se realizará la acción final de borrado si el usuario ha marcado la casilla de confirmación.

En caso de que el identificador de película no exista (bien por una modificación mal intencionada de la petición HTTP o bien porque se haya borrado en el transcurso) o no sea numérico debe mostrarse un mensaje de error apropiado, por ejemplo:

ACCION BORRAO DE UNA PELICULA

Esta acción será POST y el enrutador tendrá que captura la acción borrar_pelicula_confirmacion_XYZ que provendrá del formulario del paso anterior. Si dicha ruta no es POST el enrutador ejecutará otro método controlador donde se mostrará un error en la acción.

Esta acción (`borrar_pelicula_confirmacion_XYZ`) tendrá su método controlador propio donde se verificará que el usuario ha marcado la casilla de verificación. Si el usuario ha marcado la casilla de verificación entonces se procederá a realizar la acción de borrado.

En caso de se se produzca algún error, deberá mostrarse un mensaje adecuado al error producido:

IMPORTANTE: todas las clases se cargarán por autoloading PSR-4, en ningún momento se realizará `include` o `require`. La única situación donde se podrá hacer uso de `include` o `require` será para cargar información de configuración de acceso a la base de datos desde el archivo `index.php`.

1.5.- Ejercicio 5: documentación y prueba.

En este ejercicio vamos a documentar y probar la aplicación.

Probar el modelo

OPCION 01

Crea un script de prueba del modelo (`prueba_modelo.php`) accesible vía (`http://localhost/dwes04/prueba_modelo.php`), que cargue las clases creadas usando el archivo `vendor/autoload.php` generado por *composer*. En este script deberás verificar que las operaciones del modelo (todas) se realizan de forma adecuada en la base de datos y que los métodos `set/get` funcionan como se esperan.

OPCION 02 (REQUIERO INVESTIGACION AUTÓNOMA)

Investiga sobre el uso de PHPUnit. Instálalo a través de *composer* como dependencia de desarrollo e implementa tests para verificar que las clases del modelo funcionan adecuadamente.

Documenta las clases del modelo y el controlador

Añade comentarios PHPDoc a las clases y métodos. El siguiente ejemplo es un comentario PHPDoc a nivel de clase:

```
/**
```

```
* Clase que representa un ejemplo de documentación en PHP.
```

```
*
```

```
* Descripción detallada de la clase y su propósito.
```

```
*
```

```
* @author Tu Nombre
```

```
*/
```

```
class MiClase {
```

```
}
```

Y el siguiente ejemplo es un comentario a nivel de método:

```
/**
```

```
* Realiza .... --Descripción de la operación---.
```

```
*
```

```
* @param int $parametro1 Descripción del primer parámetro.
```

```
* @param string $parametro2 Descripción del segundo parámetro.
```

```
* @return bool True si la operación fue exitosa, false de lo contrario.
```

```
*/
```

```
public function realizarOperacion($parametro1, $parametro2) {
```

```
}
```